Unit I: Problem Solving, Programming, and Python Programming

Section 1: General Problem Solving Concepts

1.1 Problem Solving in Everyday Life

Definition:
Problem-solving is the cognitive process of searching through a space of possible solutions to reach a specific goal. In everyday life, we do this subconsciously.

Formal Definition:
A systematic approach to:

1. Defining a problem (what is wrong)
2. Determining the cause
3. Identifying, prioritizing, and selecting alternatives
4. Implementing a solution

6-Step Problem-Solving Cycle:

Step	Description	Example
1. Identify the Problem	Recognize the issue	"My car won't start."
2. Analyze the Problem	Understand the cause	"Lights turn on, engine doesn't crank."
3. Generate Potential Solutions	List possible ways to fix	Jumpstart battery, check fuel, call mechanic
4. Select the Best Solution	Choose most practical	Jumpstart battery (quickest & cheapest)
5. Implement the Solution	Apply the solution	Connect jumper cables and start the car
6. Evaluate Results	Check if problem solved	Car starts → success; else → repeat step 2

Tip: If the solution fails, loop back to analysis.

1.2 Types of Problems

1. Algorithmic Problems

- Have a precise step-by-step solution.
- Guaranteed to give the correct result.
- **Examples:**
    - Checking if a number is even or odd
    - Following a cake recipe exactly

2. Heuristic Problems

- Use trial-and-error, intuition, or approximation.
- Perfect solution may be impossible or slow to compute.
- **Examples:**
    - Chess game strategies
    - Spam detection in emails

1.3 Problem Solving with Computers

- **Key Idea:** Computers do not think; they execute instructions (algorithms).
- **Programmer's Role:** Bridge human problem → computer-executable steps.

Process:

1. **Problem Definition:** Clearly state input & output
2. **Algorithm Design:** Plan logic (flowchart/pseudocode)
3. **Coding:** Translate logic into a programming language
4. **Testing/Debugging:** Verify output against test cases

1.4 Difficulties with Problem Solving

Difficulty	Explanation	Example
Ambiguity	Computers need precise instructions	"Make it look nice" → "Set background color to #FFFFFF"
Syntactic Burden	Beginners focus too much on syntax	Missing semicolon → error
Conceptual Load	Handling variables, loops, logic together	Calculating area while understanding loops
Logic Errors	Code runs but results are wrong	Area = 2*r instead of Area = r*r

1.5 Top-Down Design (Stepwise Refinement)

Definition: Breaking a complex problem into smaller, manageable sub-tasks.

Methodology:

1. Start with main goal (Level 0)
2. Divide into major sub-tasks (Level 1)
3. Divide further into detailed tasks (Level 2)
4. Continue until tasks are simple enough to code

Advantages:

- Reduces complexity
- Enables parallel development
- Reusable modules (e.g., calculateTax() function)

Section 2: Problem Solving Strategies

2.1 Algorithms

Definition: Finite sequence of instructions to perform a task.

Characteristics of a Good Algorithm:

- **Unambiguous:** Clear steps
- **Input/Output:** Must have defined inputs & at least one output
- **Finiteness:** Terminates after finite steps
- **Feasibility:** Can be executed with available resources

2.2 Flowcharts

Purpose: Visual representation of algorithm for easy understanding.

Symbol	Shape	Meaning
Oval		Start/Stop
Parallelogram		Input/Output
Rectangle		Processing/Calculations
Diamond		Decision (Yes/No)
Arrow		Flow of control

Example: Flowchart for finding largest of 2 numbers.

2.3 Divide and Conquer

Strategy: Break problem into smaller subproblems, solve each, combine results.

Steps:

1. Divide → Split into subproblems
2. Conquer → Solve subproblems (often recursively)
3. Combine → Merge results for final solution

Example: Merge Sort algorithm

Definition

Merge Sort is a **Divide and Conquer** algorithm that:

1. Divides the array into two halves.
2. Recursively sorts each half.

3. Merges the two sorted halves into a single sorted array.

Characteristics:

- Time Complexity: O(n log n)
- Space Complexity: O(n) (requires temporary arrays)
- Stable sort (preserves order of equal elements)
- Works well for large datasets

Steps of Merge Sort

1. **Divide**: Split the array into two halves.
2. **Conquer**: Recursively sort each half.
3. **Combine**: Merge the two sorted halves into a single sorted array.

Example

Suppose we have an array:

arr = [38, 27, 43, 3, 9, 82, 10]

Step 1: Divide

[38, 27, 43, 3] [9, 82, 10]

Step 2: Divide further

[38, 27] [43, 3] [9, 82] [10]

Step 3: Divide until single element

[38] [27] [43] [3] [9] [82] [10]

Step 4: Merge and sort

[27, 38] [3, 43] [9, 82] [10]

Step 5: Merge again

[3, 27, 38, 43] [9, 10, 82]

Step 6: Final merge

[3, 9, 10, 27, 38, 43, 82]

Section 3: Basics of Python Programming

3.1 Features of Python

- **Interpreted:** Executes code line by line → easy testing
- **Dynamically Typed:** Variable types are inferred
- x = 10 # Python knows x is integer
- **Platform Independent:** Runs on Windows, Linux, Mac
- **Case Sensitive:** Number ≠ number

3.2 History and Future

- **Origin:** 1980s, Guido van Rossum, Netherlands
- **Milestones:**
    - Python 2.0 (2000): Garbage collection, list comprehensions
    - Python 3.0 (2008): Not backward-compatible, fixes design flaws
- **Future of Python**

Python is highly future-proof because of its popularity, versatility, and strong ecosystem. Its main areas of use in the future include:

1. Artificial Intelligence (AI) & Machine Learning
    - Python has rich libraries for AI and ML like TensorFlow, PyTorch, scikit-learn.
    - Used by companies like OpenAI, Google DeepMind for deep learning projects.
    - Applications: Image recognition, NLP (chatbots, translation), self-driving cars.
2. Big Data
    - Handles large datasets efficiently with libraries like Pandas, NumPy, Dask, PySpark.
    - Used for data analysis, predictive modeling, data visualization.
    - Applications: E-commerce analytics, stock market predictions, research datasets.

3. Cloud Computing
    - Python is supported natively on major cloud platforms: AWS Lambda, Google Cloud Functions, Azure.
    - Enables serverless computing, automation, and scalable web services.
    - Applications: Backend APIs, cloud-based data pipelines, SaaS platforms.

3.3 Programming Paradigms in Python

- **Procedural:** Code grouped in functions (scripts, automation)
- **Object-Oriented (OOP):** Code grouped as objects (large systems)
- **Functional:** Uses functions, avoids state changes

3.4 Features of Object-Oriented Programming (OOP)

Pillar	Meaning	Example
Encapsulation	Data hiding	__privateVar
Abstraction	Hide implementation, show interface	Drive a car via steering wheel
Inheritance	Reuse code	Student inherits from Person
Polymorphism	Same interface, different behavior	+ adds numbers, concatenates strings

---

3.5 Applications of Python

- **GUI Apps:** Tkinter, PyQt, Kivy
- **Web Scraping:** BeautifulSoup, Scrapy
- **Embedded Systems:** MicroPython on microcontrollers
- **Business Apps:** ERP, e-commerce systems
- **Audio/Video Apps:** Backend analytics like Spotify